MATERIAL DIDÁCTICO · PROGRAMACIÓN

UNIDAD DIDÁCTICA 02

Arrays, matrices y cadenas de texto

JavaScript · Almacenamiento, recorridos y tratamiento de datos

TEORÍA · EXPLICACIONES · EJEMPLOS GUIADOS

GUÍA DE LECTURA

Índice de contenidos

Seis bloques progresivos, desde las operaciones con arrays hasta los proyectos guiados.

1. Introducción a los Arrays

1.1. Definición de un array

1.2. Características y propiedades de los arrays en JavaScript

1.3. Creación de un array en JavaScript

2. Acceso y Modificación de Elementos en un Array

2.1. Acceder a elementos individuales

2.2. Acceso al último elemento de un array

2.4. Añadir elementos a un array

2.5. Eliminar elementos de un array

3. Recorrido de Arrays

3.1. Usando bucles for y while

3.2. Iteración con el bucle for...of

4. Introducción a Matrices en JavaScript

4.1. ¿Qué es una Matriz?

4.2. Cómo Acceder a Elementos en una Matriz

4.5. Recorrer una Matriz

5. Manejo de Cadenas de Texto (Strings) en JavaScript

5.1. Cómo Crear una String

5.2. Cómo Juntar Cadenas de Texto (Concatenación)

5.7. Buscar Texto en una String

6. Talleres guiados: de la explicación a la práctica

6.1. Taller guiado: gastos de una semana (arrays)

6.2. Taller guiado: mapa de temperaturas (matrices)

6.3. Taller guiado: normalización de un nombre (strings)

1. Introducción a los Arrays

1.1. Definición de un array

En JavaScript, un **array** es un tipo de estructura de datos ampliamente utilizada para almacenar colecciones de elementos, los cuales pueden ser valores primitivos (números, cadenas de texto, booleanos, etc.) u objetos más complejos. Un array permite gestionar conjuntos de datos relacionados bajo una sola variable, facilitando operaciones de acceso, búsqueda, modificación y eliminación de sus elementos.

Los arrays en JavaScript son **dinámicos** y **heterogéneos**. Esto significa que:

1. **Dinámicos**: A diferencia de otros lenguajes como C o Java, en JavaScript no es necesario especificar el tamaño del array al crearlo. Los arrays pueden crecer o decrecer dinámicamente según sea necesario, añadiendo o eliminando elementos sin necesidad de una redefinición explícita.
2. **Heterogéneos**: Los arrays en JavaScript no imponen restricciones sobre los tipos de datos que almacenan. Un solo array puede contener elementos de diferentes tipos de datos, lo que añade flexibilidad pero requiere cuidado adicional para evitar inconsistencias lógicas.

Ejemplo básico de un array heterogéneo:

```javascript
let mixto = [42, "Hola", true, {nombre: "Juan"}, [1, 2, 3]]
console.log(mixto)
// Salida: [42, "Hola", true, {nombre: "Juan"}, [1, 2, 3]]
```

En este ejemplo, el array `mixto` contiene un número, una cadena de texto, un booleano, un objeto y otro array, lo cual refleja la naturaleza heterogénea de los arrays en JavaScript.

1.2. Características y propiedades de los arrays en JavaScript

Los arrays en JavaScript son un tipo especial de objeto. Aunque comparten algunas características con los objetos, los arrays están optimizados para el almacenamiento y la manipulación secuencial de datos, lo que los hace ideales para situaciones en las que es necesario trabajar con colecciones de datos ordenados.

Índices y longitud

Cada elemento en un array tiene un **índice** asociado, que determina su posición dentro del array. Estos índices son **cero-basados**, lo que significa que el primer elemento de un array tiene el índice 0, el segundo el índice 1, y así sucesivamente. Este es un detalle crucial a tener en cuenta cuando se trabaja con arrays, especialmente en bucles y funciones de manipulación de arrays.

Además, los arrays tienen una propiedad `length` que refleja el número de elementos presentes en el array. Esta propiedad se actualiza automáticamente cuando se añaden o eliminan elementos, y puede ser utilizada tanto para obtener la longitud actual del array como para truncarlo, ya que se puede modificar directamente.

Ejemplo de uso de índices y longitud:

```javascript
let frutas = ["manzana", "banana", "naranja"]
console.log(frutas[0])  // 'manzana' (primer elemento, índice 0)
console.log(frutas.length)  // 3 (cantidad de elementos en el array)
```

1.3. Creación de un array en JavaScript

Existen varias formas de crear un array en JavaScript. La habitual y recomendada es emplear **literales de array** (`[]`); también existe el constructor `Array`.

1.3.1. Creación de arrays mediante literales

La forma más directa y preferida para crear arrays en JavaScript es usando la **sintaxis literal** de corchetes `[]`. Este método es conciso y eficiente tanto en términos de legibilidad como de rendimiento.

Ejemplo:

```javascript
let numeros = [1, 2, 3, 4, 5]
let cadenas = ["a", "b", "c"]
```

El uso de literales es generalmente el enfoque recomendado debido a su claridad y simplicidad, además de que minimiza el riesgo de errores comunes asociados al constructor `Array`.

1.3.2. Creación de arrays con el constructor `Array()`

Alternativamente, se pueden crear arrays usando el constructor `Array()`. Este método es menos común y generalmente es utilizado cuando es necesario crear un array de un tamaño específico, aunque sin inicializar sus elementos.

Ejemplo de uso del constructor:

```javascript
let arrayVacio = new Array()  // Crea un array vacío
let arrayConLongitud = new Array(5)  // Crea un array con 5 posiciones vacías
```

En el ejemplo anterior, `arrayConLongitud` tiene una longitud de 5, pero no tiene elementos inicializados. Esto es diferente de un array en el que todos los valores son `undefined`, ya que el array no tiene propiedades asignadas a los índices.

Es importante notar que, cuando se pasa un solo número como argumento al constructor, JavaScript interpretará ese número como la longitud del array que se desea crear, y no como un elemento del array.

1.3.3. Arrays vacíos y arrays con valores inicializados

Cuando se desea crear un array vacío, es suficiente con usar `[]` o `new Array()` sin argumentos. Por otro lado, si se desean inicializar los valores de un array directamente, se puede proporcionar una lista de valores.

Ejemplo:

JAVASCRIPT

```
let vacio = []
let inicializado = [1, 2, 3]
```

2. Acceso y Modificación de Elementos en un Array

El manejo eficiente de los elementos de un array en JavaScript es fundamental para optimizar el uso de esta estructura de datos. Comprender cómo acceder a los elementos individuales, así como cómo modificarlos y gestionar su estructura, es clave para trabajar con grandes volúmenes de datos o para desarrollar funcionalidades complejas.

2.1. Acceder a elementos individuales

En JavaScript, los arrays son colecciones **ordenadas**, lo que significa que cada elemento tiene una **posición o índice** dentro del array. Los índices permiten identificar y acceder a cada elemento individual de manera eficiente.

Los arrays en JavaScript utilizan un sistema de **índices cero-basados** (0-based indexing), lo que implica que el primer elemento de un array tiene el índice `0`, el segundo el índice `1`, y así sucesivamente. Este modelo de indexación es crucial, especialmente al usar bucles para recorrer arrays o cuando se desean aplicar operaciones sobre elementos individuales.

Ejemplo de acceso a elementos:

JAVASCRIPT

```
let frutas = ['manzana', 'banana', 'naranja', 'uva']

console.log(frutas[0])  // 'manzana' (primer elemento, índice 0)
console.log(frutas[2])  // 'naranja' (tercer elemento, índice 2)
```

Cuando accedemos a un índice que está fuera del rango actual del array, JavaScript devuelve `undefined`. Esto ocurre porque no existe ningún valor asociado a ese índice.

```javascript
console.log(frutas[10])  // undefined
```

2.2. Acceso al último elemento de un array

Una operación común en arrays es acceder al **último elemento**. Aunque la forma más directa de hacerlo es utilizando el índice `array[array.length - 1]`, también existen otros enfoques para simplificar esta operación.

Acceso tradicional al último elemento

La manera más básica y explícita de acceder al último elemento de un array es usando su longitud para calcular el índice correcto. Como la indexación de arrays es cero-basada, el último elemento siempre se encuentra en el índice `length - 1`.

Ejemplo:

```javascript
let frutas = ['manzana', 'banana', 'naranja']
let ultimaFruta = frutas[frutas.length - 1]
console.log(ultimaFruta)  // 'naranja'
```

Este método es eficiente y directo, pero puede resultar menos legible en algunos contextos, especialmente cuando se trabaja con arrays grandes o en funciones complejas.

Alternativa moderna: uso de `at()`

A partir de ES2022, se introdujo el método `at()`, que permite acceder a un elemento de un array usando índices negativos. Esto significa que se puede acceder al último elemento usando `array.at(-1)` de forma más intuitiva y legible.

Ejemplo usando `at()`:

```javascript
let ultimaFrutaConAt = frutas.at(-1)
console.log(ultimaFrutaConAt)  // 'naranja'
```

El método `at()` no solo mejora la legibilidad del código, sino que también permite evitar errores comunes relacionados con el cálculo de índices, especialmente en arrays dinámicos.

2.3. Modificación de valores de un array

Modificar un valor en un array implica asignar un nuevo valor a un índice específico. Dado que los arrays en JavaScript son **mutables**, se puede cambiar directamente el valor almacenado en una posición específica del array sin necesidad de crear una nueva estructura.

El proceso es similar al acceso de un elemento, pero en lugar de solo leer el valor en el índice, le asignamos uno nuevo.

Ejemplo de modificación de elementos:

```javascript
let numeros = [10, 20, 30, 40]

numeros[1] = 25  // Cambia el valor en el índice 1 (20) por 25
console.log(numeros)  // [10, 25, 30, 40]
```

Si asignamos un valor a un índice fuera de los límites actuales, JavaScript **ampliará el array** automáticamente y dejará posiciones intermedias sin asignar (*huecos*). Al leer uno de esos huecos se obtiene `undefined`.

```javascript
numeros[10] = 100
console.log(numeros)  // [10, 25, 30, 40, <6 empty items>, 100]
```

En este ejemplo, el array ahora tiene un tamaño de 11, pero las posiciones entre los índices 4 y 9 son **huecos sin elementos asignados**. Al acceder a uno de esos índices se obtiene `undefined`, pero el hueco no es lo mismo que un elemento al que se haya asignado explícitamente `undefined`.

2.4. Añadir elementos a un array

Existen varios métodos para **agregar elementos** a un array en JavaScript, tanto para añadirlos al **final** del array como al **inicio**.

2.4.1. Añadir elementos al final con `push()`

El método más común para añadir elementos a un array es `push()`. Este método inserta uno o más elementos al **final del array** y devuelve la nueva longitud del array. Este es un proceso eficiente en términos de rendimiento, ya que no requiere la reindexación de los elementos existentes.

Sintaxis:

```javascript
array.push(elemento1, elemento2, ..., elementoN)
```

Ejemplo:

```javascript
let frutas = ['manzana', 'banana']

frutas.push('naranja')
console.log(frutas)  // ['manzana', 'banana', 'naranja']
```

Además de añadir un único elemento, `push()` también permite añadir varios elementos de forma simultánea:

```javascript
frutas.push('uva', 'mango')
console.log(frutas)  // ['manzana', 'banana', 'naranja', 'uva', 'mango']
```

2.4.2. Añadir elementos al inicio con `unshift()`

Si es necesario añadir elementos al **inicio** de un array, se puede utilizar el método `unshift()`. Este método agrega uno o más elementos al inicio del array y devuelve la nueva longitud del array. Sin embargo, este proceso es más costoso en términos de rendimiento, ya que JavaScript debe **reindexar** todos los elementos existentes del array para desplazarlos hacia la derecha.

Sintaxis:

```javascript
array.unshift(elemento1, elemento2, ..., elementoN)
```

Ejemplo:

```javascript
frutas.unshift('fresa')
console.log(frutas)  // ['fresa', 'manzana', 'banana', 'naranja', 'uva', 'mango']
```

2.5. Eliminar elementos de un array

JavaScript proporciona varios métodos para **eliminar elementos** de un array, dependiendo de si se desea eliminar elementos del inicio, del final, o de una posición específica del array.

2.5.1. Eliminar el último elemento con `pop()`

El método `pop()` elimina el **último elemento** de un array, reduce su longitud y devuelve el elemento eliminado. Este método es eficiente porque no requiere la reindexación de los demás elementos.

Sintaxis:

```javascript
array.pop()
```

Ejemplo:

```javascript
let numeros = [1, 2, 3, 4]
let eliminado = numeros.pop()
console.log(numeros)  // [1, 2, 3]
console.log(eliminado)  // 4
```

2.5.2. Eliminar el primer elemento con `shift()`

Para eliminar el **primer elemento** de un array, se utiliza el método `shift()`. Este método elimina el primer elemento y devuelve el valor eliminado, pero como los índices restantes deben ser reindexados, es menos eficiente que `pop()`.

Sintaxis:

```javascript
array.shift()
```

Ejemplo:

```javascript
let frutas = ['manzana', 'banana', 'naranja']
let primerElemento = frutas.shift()
console.log(frutas)  // ['banana', 'naranja']
console.log(primerElemento)  // 'manzana'
```

---

3. Recorrido de Arrays

Recorrer un array es una operación fundamental en el manejo de datos en JavaScript. Los arrays suelen contener conjuntos de datos, y la capacidad de iterar sobre ellos de manera eficiente es esencial para realizar análisis, transformaciones y cálculos. JavaScript proporciona múltiples formas de recorrer arrays, desde métodos tradicionales como bucles `for` y `while`, hasta métodos más modernos y declarativos como `forEach()` y el bucle `for...of`.

A continuación, exploramos en detalle las diferentes técnicas de iteración disponibles para trabajar con arrays.

3.1. Usando bucles `for` y `while`

El **bucle** `for` es una de las formas más tradicionales y utilizadas para recorrer arrays en JavaScript. Es extremadamente flexible y permite un control explícito sobre el índice y el número de iteraciones. Por otro lado, el **bucle** `while` es útil cuando el número de iteraciones depende de una condición que no está relacionada directamente con el tamaño del array.

Bucle `for`

El bucle `for` es ideal para iterar sobre arrays cuando es necesario tener control sobre el índice de cada elemento y realizar operaciones con esa información. La sintaxis estándar para recorrer un array utilizando `for` es:

Sintaxis:

```javascript
for (let i = 0 ; i < array.length ; i++) {
  // Operaciones con array[i]
}
```

- `i`: El índice que empieza desde 0 y aumenta con cada iteración.
- `array.length`: Condición que asegura que el bucle solo recorra los elementos presentes en el array.

Ejemplo de uso:

```javascript
let numeros = [10, 20, 30, 40, 50]
for (let i = 0 ; i < numeros.length ; i++) {
  console.log(`El número en el índice ${i} es: ${numeros[i]}`)
}
```

Este bucle es simple y eficiente. Se itera desde el primer índice (`0`) hasta el último (`array.length - 1`). El control sobre el índice permite realizar operaciones adicionales, como saltarse elementos, iterar en orden inverso o modificar el array durante la iteración.

Bucle `while`

El bucle `while` ejecuta un bloque de código **mientras** una condición especificada sea verdadera. Es una alternativa al bucle `for` cuando la lógica de iteración no depende de un índice, o cuando se espera que la condición de terminación pueda cambiar dinámicamente durante la ejecución.

Sintaxis:

```javascript
let i = 0
while (i < array.length) {
  // Operaciones con array[i]
  i++
}
```

Ejemplo de uso:

```javascript
let frutas = ['manzana', 'banana', 'naranja']
let i = 0
while (i < frutas.length) {
  console.log(`Fruta en índice ${i}: ${frutas[i]}`)
  i++
}
```

El bucle `while` ofrece flexibilidad adicional, como poder modificar el valor de `i` dentro del bucle según condiciones específicas. No obstante, es importante asegurarse de que la condición de parada (`i < array.length`) eventualmente se cumpla para evitar bucles infinitos.

3.2. Iteración con el bucle `for...of`

El bucle `for...of` fue introducido en ECMAScript 6 (ES2015) y es una forma sencilla y elegante de iterar sobre **valores** de arrays (y otros objetos iterables). A diferencia del bucle `for` tradicional, `for...of` está diseñado específicamente para **recorrer los valores** de un array sin preocuparse por los índices. Esto hace que sea una opción preferida cuando no se necesita acceso directo al índice y solo se desean los valores del array.

Sintaxis:

```javascript
for (let valor of array) {
  // Operaciones con valor
}
```

- `valor`: Representa el valor de cada elemento del array en la iteración.

Ejemplo de uso:

```javascript
let frutas = ['manzana', 'banana', 'naranja']
for (let fruta of frutas) {
  console.log(`Fruta: ${fruta}`)
}
```

Este bucle recorre directamente los valores del array sin exponer explícitamente los índices. Es especialmente útil cuando no se necesita realizar operaciones basadas en el índice de cada elemento.

Ventajas de `for...of`:

1. **Claridad**: Es una forma más simple de recorrer los elementos de un array cuando no se necesita trabajar con los índices.
2. **Compatibilidad con otros iterables**: `for...of` no se limita solo a arrays, también puede recorrer otros tipos de objetos iterables, como `Map`, `Set`, cadenas de texto y objetos generados por generadores.
3. **Legibilidad**: Proporciona una sintaxis más concisa y clara en comparación con el bucle `for` tradicional, lo que mejora la legibilidad del código.

Limitaciones de `for...of`:

1. **Sin índices**: No proporciona acceso directo a los índices del array. Si se requiere tanto el índice como el valor, es necesario utilizar un contador externo o recurrir al método `entries()` en combinación con `for...of`.

---

4. Introducción a Matrices en JavaScript

Imagina que una matriz en JavaScript es como una tabla o una cuadrícula en la que cada "fila" es una lista de elementos, y cada "columna" es un elemento dentro de esa lista. En lugar de tener una lista larga con un solo tipo de elementos, puedes organizar tus datos en filas y columnas para que sea más fácil de manejar. Por ejemplo, si tienes una tabla de asientos de un cine o una lista de productos en un almacén, una matriz te permite organizar todo de manera clara y estructurada.

4.1. ¿Qué es una Matriz?

Una **matriz** en JavaScript es un tipo de array (o lista) que contiene otros arrays dentro de él. Cada sub-array representa una "fila" en esta estructura, y dentro de cada fila tenemos "columnas", que son los elementos específicos en cada posición.

Pensemos en un ejemplo simple de una sala de cine. Cada fila en la matriz representará una fila de asientos, y cada asiento se representará como un elemento en esa fila. De esta forma, la estructura es más fácil de entender visualmente y de usar en nuestro código.

Ejemplo de una sala de cine con 3 filas y 4 asientos por fila:

JAVASCRIPT

```javascript
let salaDeCine = [
  ["A1", "A2", "A3", "A4"], // Fila A
  ["B1", "B2", "B3", "B4"], // Fila B
  ["C1", "C2", "C3", "C4"]  // Fila C
]
console.log(salaDeCine)
```

Esta matriz representa una sala de cine con tres filas de asientos (`A`, `B` y `C`) y cuatro asientos en cada fila. Cada posición en la matriz muestra el nombre del asiento.

4.2. Cómo Acceder a Elementos en una Matriz

Para acceder a un elemento específico en una matriz, necesitamos saber en qué fila y en qué columna se encuentra. Primero elegimos la fila (por ejemplo, la fila 1 es la segunda fila porque las filas empiezan a contarse desde 0), y luego seleccionamos la columna dentro de esa fila.

Ejemplo: Acceder al asiento B3 en nuestra matriz de cine

JAVASCRIPT

```javascript
let asientoSeleccionado = salaDeCine[1][2]  // Fila 1, columna 2
console.log(asientoSeleccionado)  // Salida: "B3"
```

En este ejemplo, hemos accedido al asiento `B3` especificando que queremos el segundo array (fila B) y el tercer elemento dentro de esa fila (asiento B3).

4.3. Cómo Modificar Elementos en una Matriz

Modificar un elemento en una matriz funciona igual que acceder a él, pero en lugar de solo leer el valor, le asignamos uno nuevo. Supongamos que queremos cambiar el estado del asiento `C2` a "Reservado".

Ejemplo: Cambiar el estado de un asiento a "Reservado"

JAVASCRIPT

```
salaDeCine[2][1] = "Reservado"  // Fila 2, columna 1 (C2)
console.log(salaDeCine)
```

Ahora, el asiento `C2` está marcado como "Reservado" en nuestra matriz.

4.4. Crear Matrices de Forma Dinámica

A veces no sabemos de antemano cuántas filas o columnas necesitaremos, como cuando el tamaño de una matriz depende de datos que obtenemos de otra fuente. En estos casos, podemos construir la matriz dinámicamente con un par de bucles.

Imaginemos que queremos organizar un **estacionamiento de un centro comercial** en tres pisos. Cada piso tiene cinco filas de espacios de estacionamiento y cada fila tiene cuatro espacios. Usaremos bucles para crear esta matriz.

Ejemplo: Crear un estacionamiento de 3 pisos con 5 filas y 4 espacios por fila

JAVASCRIPT

```
let estacionamiento = []
let totalPisos = 3
let totalFilasPorPiso = 5
let totalEspaciosPorFila = 4

for (let piso = 0; piso < totalPisos; piso++) {
  let pisoActual = []
  for (let fila = 0; fila < totalFilasPorPiso; fila++) {
    let filaActual = []
    for (let espacio = 0; espacio < totalEspaciosPorFila; espacio++) {
      filaActual.push(`Piso ${piso + 1}, Fila ${fila + 1}, Espacio ${espacio + 1}`)
    }
    pisoActual.push(filaActual)
  }
  estacionamiento.push(pisoActual)
}

console.log(estacionamiento)
```

Aquí hemos creado una **estructura tridimensional** llamada `estacionamiento`: tres pisos, con cinco filas de cuatro espacios cada uno. Es una extensión de la idea de matriz bidimensional.

4.5. Recorrer una Matriz

Para ver o procesar todos los elementos de una matriz, podemos recorrer cada fila y cada elemento dentro de esa fila usando bucles. Esto nos permite acceder a cada espacio en el estacionamiento o cada asiento en la sala de cine, por ejemplo.

Ejemplo: Recorrer el estacionamiento y mostrar cada espacio

JAVASCRIPT

```
for (let piso = 0; piso < estacionamiento.length; piso++) {
  console.log(`Piso ${piso + 1}:`)
  for (let fila = 0; fila < estacionamiento[piso].length; fila++) {
    console.log(`  Fila ${fila + 1}: ${estacionamiento[piso][fila]}`)
  }
}
```

Este código imprime todos los espacios en el estacionamiento, agrupados por piso y fila.

4.6. Operaciones Comunes con Matrices

A continuación, veremos algunas operaciones comunes que se pueden realizar con matrices en aplicaciones prácticas.

4.6.1. Verificar Disponibilidad de un Asiento

Imagina que queremos comprobar si un asiento específico en una sala de cine está disponible. Si el asiento tiene el valor "Reservado" u "Ocupado", no está disponible.

Ejemplo: Función para verificar disponibilidad de un asiento

JAVASCRIPT

```
function verificarDisponibilidad(salaDeCine, fila, columna) {
  let estadoAsiento = salaDeCine[fila][columna]
  let disponibilidad

  if (estadoAsiento === "Reservado" || estadoAsiento === "Ocupado") {
    disponibilidad = "Asiento no disponible"
  } else {
    disponibilidad = "Asiento disponible"
  }

  return disponibilidad
}

console.log(verificarDisponibilidad(salaDeCine, 2, 1))  // Salida: "Asiento no disponible"
console.log(verificarDisponibilidad(salaDeCine, 0, 3))  // Salida: "Asiento disponible"
```

En este ejemplo, la función revisa el estado del asiento y devuelve si está disponible o no.

4.6.2. Reservar un Asiento en la Sala de Cine

Podemos escribir una función que reserve un asiento, si está disponible, cambiando su valor a "Reservado". Así evitamos que alguien lo reserve más de una vez.

Ejemplo: Función para reservar un asiento

JAVASCRIPT

```
function reservarAsiento(salaDeCine, fila, columna) {
  let estadoAsiento = salaDeCine[fila][columna]
  if (estadoAsiento === "Reservado" || estadoAsiento === "Ocupado") {
    return "El asiento ya está ocupado"
  } else {
    salaDeCine[fila][columna] = "Reservado"
    return "Asiento reservado exitosamente"
  }
}

console.log(reservarAsiento(salaDeCine, 0, 1))  // Salida: "Asiento reservado exitosamente"
console.log(salaDeCine)
```

La función solo reserva el asiento si está disponible, y nos avisa si el asiento ya estaba ocupado.

4.6.3. Contar Asientos Disponibles

Para contar cuántos asientos están disponibles en la sala, podemos recorrer toda la matriz y contar aquellos que no están reservados ni ocupados.

Ejemplo: Función para contar asientos disponibles

JAVASCRIPT

```
function contarAsientosDisponibles(salaDeCine) {
  let disponibles = 0
  for (let fila of salaDeCine) {
    for (let asiento of fila) {
      if (asiento !== "Reservado" && asiento !== "Ocupado") {
        disponibles++
      }
    }
  }
  return disponibles
}

console.log(`Asientos disponibles: ${contarAsientosDisponibles(salaDeCine)}`)
```

Aquí estamos contando todos los asientos en la sala que no están reservados ni ocupados.

4.7. Ejemplos de Matrices en la Vida Real

Las matrices pueden usarse en muchas aplicaciones del mundo real, como:

1. **Mapa de un supermercado**: Cada pasillo es una fila de estantes, y cada estante es una columna en la matriz. Esto facilita la organización de productos y su búsqueda.

2. **Inventario de almacén**: Las ubicaciones en un almacén se organizan en filas y columnas de estanterías, facilitando la localización de productos.
3. **Asientos en un estadio**: Los asientos de un estadio se organizan en secciones, filas y columnas, lo cual es ideal para representarse en una matriz.

---

5. Manejo de Cadenas de Texto (Strings) en JavaScript

En JavaScript, una **string** es simplemente una **cadena de texto**. Esto significa que podemos guardar palabras, frases, nombres, mensajes, y mucho más. Trabajar con strings es muy importante en cualquier aplicación o página web, porque la mayoría de las interacciones con los usuarios implican texto de algún tipo.

JavaScript tiene muchas formas de crear y manipular strings. A continuación, exploraremos los conceptos básicos y algunos trucos útiles para que puedas manejar texto sin problemas.

5.1. Cómo Crear una String

Hay tres formas básicas de crear una cadena de texto (string) en JavaScript:

1. Usando **comillas dobles** (`"texto"`).
2. Usando **comillas simples** (`'texto'`).
3. Usando **backticks** (`` `texto` ``), también llamados *template literals*.

Los backticks son muy útiles porque te permiten insertar variables o hacer operaciones dentro del texto. Esto puede hacer que las cosas sean mucho más fáciles y claras.

Ejemplo de cómo crear strings:

JAVASCRIPT

```javascript
let mensaje = "Hola, mundo"        // Usando comillas dobles
let saludo = 'Bienvenido'           // Usando comillas simples
let nombre = "Ana"
let bienvenida = `Hola, ${nombre}!` // Usando backticks

console.log(mensaje)       // Muestra: Hola, mundo
console.log(bienvenida)    // Muestra: Hola, Ana!
```

En este ejemplo, `mensaje` usa comillas dobles, `saludo` usa comillas simples, y `bienvenida` usa backticks. Con los backticks podemos insertar variables en medio del texto usando `${nombre}`.

5.2. Cómo Juntar Cadenas de Texto (Concatenación)

A veces necesitamos **juntar varias cadenas de texto**. Esto se llama **concatenación**. Hay dos formas comunes de hacerlo:

1. **Usando el signo `+`**: Así podemos juntar texto y variables.
2. **Usando backticks**: Esto es más limpio y fácil de leer, especialmente cuando tienes muchas cosas que juntar.

Ejemplo de concatenación de strings:

JAVASCRIPT

```
let parte1 = "Hola"
let parte2 = "mundo"

// Usando el signo +
let saludoCompleto = parte1 + ", " + parte2 + "!"
console.log(saludoCompleto)  // Muestra: Hola, mundo!

// Usando backticks
let saludoAlternativo = `${parte1}, ${parte2}!`
console.log(saludoAlternativo)  // Muestra: Hola, mundo!
```

Ambas formas funcionan, pero los backticks suelen ser más claros, especialmente cuando tienes que insertar muchas palabras o frases.

5.3. Cambiar Mayúsculas y Minúsculas

Si quieres que todo el texto esté en mayúsculas o en minúsculas, JavaScript tiene dos funciones muy útiles:

- `toUpperCase()`: Cambia todo el texto a **mayúsculas**.
- `toLowerCase()`: Cambia todo el texto a **minúsculas**.

Ejemplo de cambio a mayúsculas y minúsculas:

JAVASCRIPT

```
let texto = "JavaScript es Genial!"
console.log(texto.toUpperCase())  // Muestra: JAVASCRIPT ES GENIAL!
console.log(texto.toLowerCase())  // Muestra: javascript es genial!
```

5.4. Extraer Parte de una String

Si necesitas **extraer una parte de una cadena de texto**, puedes usar el método `slice()`. Con `slice()`, solo tienes que decirle a JavaScript desde dónde quieres cortar y hasta dónde.

Ejemplo usando `slice()`:

```javascript
let texto = "Aprender JavaScript es divertido"
let parte = texto.slice(9, 19)
console.log(parte)  // Muestra: JavaScript
```

En este caso, `slice(9, 19)` toma el texto desde la posición 9 hasta la 18 (el último número no se incluye).

5.5. Reemplazar Texto en una String

Si quieres **cambiar una palabra o frase en una cadena de texto**, puedes usar `replace()`. Esto solo cambia la primera coincidencia que encuentra.

Ejemplo de reemplazo de texto:

```javascript
let frase = "JavaScript es divertido"
let nuevaFrase = frase.replace("divertido", "poderoso")
console.log(nuevaFrase)  // Muestra: JavaScript es poderoso
```

Aquí cambiamos la palabra "divertido" por "poderoso". Si quieres cambiar todas las coincidencias de una palabra, puedes usar `replaceAll()`.

5.6. Dividir una String en Partes

El método `split()` permite **dividir una cadena de texto en varias partes**, usando un carácter de separación, como una coma o un espacio.

Ejemplo usando `split()`:

```javascript
let lista = "manzana, pera, mango, banana"
let frutas = lista.split(", ")
console.log(frutas)  // Muestra: ["manzana", "pera", "mango", "banana"]
```

Aquí `split()` divide el texto en cada coma y espacio, y crea un array (lista) con cada palabra.

5.7. Buscar Texto en una String

JavaScript tiene varios métodos para **buscar palabras o frases dentro de una cadena de texto**:

- `indexOf()`: Devuelve la posición de la primera aparición de una palabra, o `-1` si no la encuentra.
- `includes()`: Devuelve `true` si la palabra está en el texto, y `false` si no está.
- `startsWith()`: Verifica si el texto **empieza** con una palabra específica.
- `endsWith()`: Verifica si el texto **termina** con una palabra específica.

Ejemplo de búsqueda:

```javascript
let texto = "Aprender JavaScript es divertido"

// Buscar la posición de "JavaScript"
let posicion = texto.indexOf("JavaScript")
console.log(posicion)  // Muestra: 9

// Verificar si incluye "es"
console.log(texto.includes("es"))  // Muestra: true

// Verificar si empieza con "Aprender"
console.log(texto.startsWith("Aprender"))  // Muestra: true

// Verificar si termina con "divertido"
console.log(texto.endsWith("divertido"))  // Muestra: true
```

Estos métodos son muy útiles cuando necesitas saber si una string contiene ciertas palabras o si empieza o termina de una forma específica.

5.8. Eliminar Espacios en Blanco

Para **eliminar espacios en blanco al principio y al final de una string**, usamos el método `trim()`. Esto es muy útil cuando trabajamos con texto que viene de formularios de usuario y puede tener espacios extra.

Ejemplo usando `trim()`:

```javascript
let entrada = "   JavaScript   "
let entradaLimpia = entrada.trim()
console.log(entradaLimpia)  // Muestra: "JavaScript"
```

5.9. Contar la Longitud de una String

Para saber **cuántos caracteres tiene una string**, incluyendo espacios y símbolos, usamos la propiedad `length`.

Ejemplo:

```javascript
let texto = "Hola, mundo"
console.log(texto.length)  // Muestra: 11
```

Esto es útil cuando quieres limitar la cantidad de caracteres en una entrada o verificar que un texto tiene una longitud específica.

5.10. Ejemplo Práctico: Crear un Mensaje de Bienvenida

Vamos a usar varios de estos métodos para hacer un **mensaje de bienvenida personalizado**. Supongamos que tenemos el nombre de un usuario que puede tener espacios en blanco o estar en minúsculas. Queremos limpiar el nombre, ponerlo en mayúsculas y crear un mensaje de bienvenida.

Ejemplo completo:

JAVASCRIPT

```
function crearMensajeBienvenida(nombreUsuario) {
  // 1. Limpiar el nombre para quitar espacios al principio y al final
  let nombreLimpio = nombreUsuario.trim()

  // 2. Convertir el nombre a mayúsculas
  let nombreEnMayusculas = nombreLimpio.toUpperCase()

  // 3. Crear el mensaje de bienvenida usando backticks
  let mensajeBienvenida = `¡BIENVENIDO, ${nombreEnMayusculas}!`

  // 4. Retornar el mensaje final
  return mensajeBienvenida
}

// Probamos la función con un nombre
let nombre = "   ana   "
console.log(crearMensajeBienvenida(nombre))  // Muestra: ¡BIENVENIDO, ANA!
```

En este ejemplo:

1. Usamos `trim()` para quitar los espacios extra.
2. Usamos `toUpperCase()` para poner el nombre en mayúsculas.
3. Usamos backticks para juntar el mensaje completo con el nombre.

---

6. Talleres guiados: de la explicación a la práctica

Los siguientes ejemplos muestran una forma de **descomponer el problema en pasos**. Lee el enunciado, predice el resultado, ejecuta el código y modifica los datos para comprobar qué ocurre. Se trata de **ejemplos explicados de la unidad**, no de soluciones de las relaciones de ejercicios.

6.1. Taller guiado: gastos de una semana (arrays)

Objetivo: guardar gastos diarios y calcular el total y los días que superaron 10 €.

Paso 1. Define los siete gastos dentro de un array. **Paso 2.** Recorre sus posiciones con `for`. **Paso 3.** Acumula el gasto y cuenta los días que superan el umbral. **Paso 4.** Muestra los resultados al terminar el recorrido.

JAVASCRIPT

```
const gastos = [8, 12, 4, 15, 7, 20, 5]
let total = 0
let diasSuperioresA10 = 0

for (let i = 0; i < gastos.length; i++) {
  total += gastos[i]
  if (gastos[i] > 10) {
    diasSuperioresA10++
  }
}

console.log("Gasto total: " + total + " €")
console.log("Días con más de 10 €: " + diasSuperioresA10)
```

Comprueba: cambia el primer gasto por `30`; después añade un octavo valor. Observa que el recorrido sigue funcionando porque utiliza `gastos.length`.

6.2. Taller guiado: mapa de temperaturas (matrices)

Objetivo: calcular la suma de las temperaturas almacenadas en una tabla de dos filas y tres columnas.

Paso 1. Cada array interior representa una fila. **Paso 2.** El primer bucle selecciona las filas. **Paso 3.** El segundo recorre las columnas de la fila actual. **Paso 4.** Cuenta las celdas para obtener la media.

JAVASCRIPT

```
const temperaturas = [
  [18, 19, 21],
  [17, 20, 22]
]
let suma = 0
let cantidad = 0

for (let fila = 0; fila < temperaturas.length; fila++) {
  for (let columna = 0; columna < temperaturas[fila].length; columna++) {
    suma += temperaturas[fila][columna]
    cantidad++
  }
}

console.log("Temperatura media: " + suma / cantidad)
```

Comprueba: añade una tercera fila y observa cómo se adapta el recorrido sin modificar los límites de los bucles.

6.3. Taller guiado: normalización de un nombre (strings)

Objetivo: recibir un nombre con espacios sobrantes, normalizarlo y comprobar si incluye una letra.

Paso 1. Limpia los espacios de los extremos con `trim()`. **Paso 2.** Convierte el resultado a minúsculas con `toLowerCase()`. **Paso 3.** Busca una secuencia con `includes()`. **Paso 4.** Muestra los resultados.

```javascript
const nombreOriginal = "   María López   "
const nombreLimpio = nombreOriginal.trim()
const nombreNormalizado = nombreLimpio.toLowerCase()

console.log("Nombre: " + nombreLimpio)
console.log("Normalizado: " + nombreNormalizado)
console.log("Contiene 'lo': " + nombreNormalizado.includes("lo"))
```

Comprueba: prueba con un nombre distinto, con varias mayúsculas y sin espacios sobrantes. Recuerda que los métodos de strings generan nuevas cadenas; no alteran la original.

6.4. Antes de comenzar las relaciones

- **R1:** creación, acceso, inserción y borrado en arrays; enunciados cortos con instrucciones concretas.
- **R2:** problemas contextualizados y programas con menú; combinar búsquedas, acumuladores y arrays paralelos.
- **R3:** problemas algorítmicos con arrays, primero con el código principal y después, como ampliación, mediante funciones que **devuelven resultados**.
- **R4:** matrices y recorridos por filas y columnas.
- **R5:** operaciones con strings y combinaciones de texto con arrays.
- **R6:** repaso final sin guía paso a paso, integrando contenidos de la unidad.